A Multiparty Authentication Scheme Based on Aggregate Certificateless Signature for Smart Healthcare

Yousheng Zhou, Longjie Li, Xiangjian Zuo, and Yuanni Liu

***Abstract*—Smart healthcare refers to the innovative medical model that enhances the efficiency and quality of medical services through the use of modern information technology. While smart healthcare brings convenience to both patients and doctors, it also raises issues of medical data security and privacy protection. In smart healthcare, secure identity authentication not only requires the verification of user identities but also ensures that user identity information is not leaked. At the same time, it requires an efficient authentication process to cope with resource-constrained situations. Existing schemes have addressed the issues of identity authentication and the confidentiality of identity information, but they have significant overhead and are not suitable for resource-constrained situations. Additionally, the schemes themselves are vulnerable to attacks. To address these issues, we propose a multiparty authentication scheme for smart healthcare. First, based on certificateless public key cryptography (CLPKC), a certificateless signature scheme (CLS) is introduced, which eliminates the need for complex bilinear pairings and map-to-point hash function computations, enabling batch authentication for multiple users. Second, the scheme incorporates the user's contextual environment, ensuring secure user identity authentication only under safe access conditions. Finally, blockchain technology is utilized to record access logs, enabling user traceability. Security analysis proves that the proposed scheme can meet security requirements such as conditional privacy protection, unlinkability, resistance to replay attacks, and resistance to collusion attacks. Compared with existing schemes, this scheme has lower computational overhead and communication costs, making it more suitable for lightweight multiparty authentication in smart healthcare scenarios.**

***Index Terms*—Batch authentication, certificateless public key cryptography (CLPKC), conditional privacy protection, identity authentication, smart healthcare.**

Received 15 April 2025; revised 20 May 2025; accepted 12 June 2025. Date of publication 16 June 2025; date of current version 25 August 2025. This work was supported in part by the Nation Key Research and Development Program of China under Grant 2023YFF0905300; in part by the National Natural Science Foundation of China under Grant 62272076; in part by the Natural Science Foundation of Chongqing, China, under Grant CSTB2022NSCQ-MSX0038; and in part by the Science and Technology Research Program of Chongqing Municipal Education Commission under Grant KJQN202200625. *(Corresponding authors: Xiangjian Zuo; Yousheng Zhou.)*

Yousheng Zhou, Xiangjian Zuo, and Yuanni Liu are with the School of Cyber Security and Information Law, Chongqing University of Posts and Telecommunications, Chongqing 400065, China (e-mail: zhouys@cqupt.edu.cn; zuoxj@cqupt.edu.cn; liuyn@cqupt.edu.cn).

Longjie Li is with the College of Computer Science and Technology, Chongqing University of Posts and Telecommunications, Chongqing 400065, China (e-mail: Llj1149609979@163.com).

Digital Object Identifier 10.1109/JIOT.2025.3580177

I. Introduction

With the rapid development of digital information technology and the growing demand for medical service levels, smart healthcare has emerged [1]. Smart healthcare refers to the innovative medical model that enhances the efficiency and quality of medical services through the use of modern information technology. It makes medical services more convenient and rapid. Telemedicine and video consultations break through the geographical restrictions of traditional medical services, allowing patients to enjoy professional medical services at home, which improves the convenience of seeking medical care [2]. At the same time, big data analysis and artificial intelligence technologies in smart healthcare can provide personalized treatment plans for each patient [3]. The sharing of medical resources can break the information silos between hospitals, optimize the allocation of medical resources, and enhance the overall level of diagnosis and treatment. By utilizing medical sensor nodes attached to patients, data can be transmitted to doctors for real-time feedback. However, since these medical records are often transmitted over public channels, attackers can easily eavesdrop, tamper with, or forge medical data. This can lead to incorrect judgments by medical professionals, posing severe threats to patients' lives [4]. Additionally, In smart healthcare systems, medical personnel frequently access the system to retrieve patient-related information, while patients can also use the system to check their own health status. However, if the system is accessed by unauthorized users, it may lead to serious security issues such as data leakage or tampering. Therefore, ensuring the integrity, validity, and availability of patients' medical data and establishing a secure and reliable authentication mechanism are essential to ensure the proper functioning of smart healthcare systems.

During the access process, users are required to submit identity information; for doctors and nurses who use the system frequently, each access involves the exchange of identity-related data, which significantly increases the risk of sensitive information leakage. Once identity information is compromised, it can lead to a range of severe consequences, including identity theft, unauthorized access to medical records, and the misuse of personal health data, all of which pose serious threats to patient privacy and healthcare security. These issues highlight the urgent need to implement robust privacy protection mechanisms within smart healthcare systems. Moreover, to address malicious user behavior, the system should also

2327-4662 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 18,2026 at 09:20:19 UTC from IEEE Xplore. Restrictions apply.

incorporate effective traceability mechanisms to support the resolution of medical disputes, misdiagnosis accountability, and security incident investigations. As such, traceability is not only a fundamental component for establishing system accountability but also a critical security requirement in smart healthcare environments. Therefore, protecting user privacy and enabling the traceability of user actions have become critical issues in smart healthcare.

Digital signatures, as an important tool to ensure the security of electronic data transmission, can not only verify identity but also ensure the integrity of data. At the same time, digital signatures have nonrepudiation, support public verification, and meet the security needs for various data sharing scenarios [5]. Therefore, owing to its prominent security features, digital signature has become a key technology widely adopted in the field of smart healthcare. However, public key infrastructure (PKI)-based schemes [6], [7], as the number of users increases, the number of certificates required for users also increases, leading to a series of issues such as certificate generation, storage, distribution, and revocation, making certificate management extremely complex. The introduction of identity-based signature schemes has solved the certificate management issues of PKI-based signatures [8], [9]. However, since identity-based signatures are all generated by the private key generator (PKG), the PKG holds all user private keys, which means it can impersonate users to generate signatures for messages, once the security of the PKG is compromised, all associated users are at risk, which is the key escrow problem.

In certificateless cryptographic systems, since the KGC only generates a partial private key and the full private key is derived from both this partial key and the user's secret value, the KGC does not have full control over the private key. As a result, certificateless schemes effectively mitigate the key escrow problem inherent in identity-based signature schemes. Researchers have made significant progress in the study of certificateless signature schemes (CLSs). Considering that a large number of sensor devices in smart healthcare are resource-constrained with limited computational capabilities, schemes that employ bilinear pairings and hash-to-point functions are not suitable due to their high computational overhead [10], [11], [12]. Meanwhile, some ordinary CLSs without pairing operations also fail to meet the requirements, as their computational cost increases with the number of signatures. The introduction of aggregate signature technology has significantly improved authentication efficiency. However, these schemes often overlook collusion attacks during the aggregation process, allowing malicious medical sensor nodes to forge seemingly legitimate but actually illegitimate aggregate signatures [13], [14], [15], [16], [17]. Consequently, doctors may receive falsified medical data from compromised sensor nodes, leading to erroneous diagnoses and posing severe threats to patient health. Therefore, designing an efficient and secure solution in the context of smart healthcare remains a challenge.

To facilitate the sharing of medical data for better user services, ensuring the security of healthcare data and user authentication is crucial. Additionally, to accommodate certain resource-constrained devices or environments, more efficient solutions are required. Based on these motivations, this article proposes a multiparty authentication scheme based on certificateless signatures to achieve efficient authentication of user access in smart healthcare, while also considering the contextual environment of user interactions. The main contributions of this article are as follows.

1) We propose a certificateless aggregatable signature scheme that avoids the use of bilinear pairings and map-to-point hash functions, resulting in lower computational cost during signature verification. Moreover, when multiple users' signatures are received, the scheme supports signature aggregation for batch authentication, thereby enabling multiparty authentication. Additionally, by introducing randomness during aggregation, the scheme achieves resistance to collusion attacks.
2) We combine our CLS with blockchain and smart contracts and apply it to the authentication scenario in smart healthcare. When a user needs to access the system, the user's environmental context and biometric context are extracted to calculate the corresponding security weight, assess the security of the user's context, and simultaneously authenticate the user's identity using the certificateless signature.
3) We conducted a security analysis of our CLS. Through both formal and informal security analyzes, we proved that the scheme can resist attacks such as replay attacks, collusion attacks, Type-I and Type-II adversarial attacks. Additionally, it provides unlinkability, traceability, and also achieves conditional privacy protection.
4) We performed a performance analysis and experimental simulation of the proposed CLS, and compared it with other CLSs. The results demonstrate that our scheme offers higher efficiency in signature authentication while also having lower communication overhead.

The remainder of this article is organized as follows. Section II presents the related work. Section III of this article introduces the preliminaries. Section IV introduces our proposed scheme. Section V conducts both formal and informal security analyzes of the proposed CLS. Section VI provides a performance analysis. Finally, Section VII summarizes this article.

II. Related Work

Certificateless signatures, similar to traditional identity-based signatures, also require a key generation center (KGC). However, the KGC only generates a partial private key corresponding to the user's identity, while the user selects a secret value as their private key. Since the KGC does not know the user's secret value, it cannot forge a legitimate signature on behalf of the user, effectively addressing the key escrow problem. Due to their excellent properties, certificateless signatures have been widely applied in scenarios such as industrial IoT [18], [19], VANETs [20], [21], and healthcare [15], [22]. Moreover, CLSs combined with blockchain have been proposed to enhance their applicability [19], [20]. Al-Riyami and Paterson [23] first proposed the CLS, and subsequently, various CLSs were proposed by researchers,

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 18,2026 at 09:20:19 UTC from IEEE Xplore. Restrictions apply.

Shim [24] proposed a CLS under the Diffie–Hellman assumption in the standard model and studied its practical feasibility. However, Shim's scheme is not secure. Yang et al. [25] pointed out the flaws in Shim et al.'s scheme and provided specific attack methods. They also improved Shim et al.'s scheme by proposing a blockchain-based CLS scheme. He et al. [20] proposed a batch verification certificateless signature algorithm based on the SM2 algorithm. Wang et al. [19] proposed a new pairings-free certificateless scheme that utilizes blockchain technology and smart contracts to construct a reliable and efficient CLS. This scheme offers high verification efficiency and low communication costs. However, Shim [18] pointed out the security issues in Wang et al.'s scheme. In Wang et al.'s scheme, only the private key is used during signing, without selecting a random value. Therefore, anyone can forge a signature using two valid signatures. Shim et al. also proposed a new CLS to address the issues with Wang et al.'s scheme. Wu et al. [21] proposed a batch-verifiable CLS for vehicle-to-vehicle (V2V) communication, which uses secure symmetric key encryption schemes and pseudonym techniques to achieve conditional privacy. In [26], a new certificateless aggregate signature scheme for VANETs with conditional privacy protection was proposed to address a series of security challenges related to vehicle privacy protection, authenticity of transmitted information, and bandwidth limitations. This scheme uses full aggregation technology to reduce computational load and bandwidth usage. However, later Yang et al. [27] pointed out issues with the scheme in [26], providing two types of forgery attacks, and proposed an improved signature method and blockchain-based transaction solution. Lin et al. [28] proposed a batch-verifiable revocable certificateless signature (RCL-SS) scheme that can be applied to cloud storage systems where users store sensitive information. Users maintain the overall integrity of files through verifiable signatures. Batch verification ensures efficiency when verifying a large number of message blocks, making it suitable for large-scale cloud environments. However, the above schemes, due to the use of bilinear pairings and hash-to-point functions, involve extensive complex computations during signature verification, resulting in significant computational overhead in the verification phase.

To address the issue of high overhead caused by bilinear pairings, many CLSs that do not use bilinear pairings have been proposed by researchers. Jia et al. [29] proposed a CLS for Internet of Things Deployment and demonstrated its effectiveness and security through experiments. However, Du et al. pointed out in [30] that Jia et al.'s scheme cannot resist attacks from Type-I adversaries. In response, Du et al. proposed a CLS based on elliptic curve cryptography, proving that the scheme cannot be forged by two types of super adversaries. Furthermore, Xiang et al. [31] noted that Jia et al.'s scheme is vulnerable to malicious but passive strong KGC attacks, which means it cannot resist Type-II adversary attacks. They also proposed a CLS without bilinear pairings and proved its security. However, not long after, Ma et al. [32] pointed out the flaws in the CLSs of Du et al. [30] and Xiang et al. [31], and providing specific attack methods. They also proposed a secure CLS that does not use bilinear pairings and applied it to identity authentication protocols. Yang et al. [33], after conducting specific attacks on Wang et al.'s scheme [19], demonstrated that Wang et al.'s scheme is incomplete. They improved Wang et al.'s decentralized KGC CLS and proposed a decentralized KGC CLS that does not require bilinear pairings. Zhan et al. [15] improved a certificateless aggregate signature scheme for medical wireless sensor networks that does not use pairings, proving in the random oracle model that the improved CLAS scheme is resistant to chosen-message attacks. However, Yang et al. [22] pointed out that the ZH-CLAS scheme in [15] cannot resist public key substitution attacks. They improved the scheme and proposed a certificateless aggregate signature scheme to address this issue. To ensure the security of vehicle communications in vehicular networks and address the key escrow problem, Li et al. [34] proposed a reliable and secure vehicular ad hoc network CLAS protocol, RelCLAS. This protocol stores the partial public keys selected during the vehicle registration phase in a key accumulator to resist malicious KGCs, while also supporting aggregate signature authentication to improve efficiency. Liu et al. [35] proposed an efficient and secure certificateless blind signature (CLBS) algorithm that does not use bilinear pairings and map-to-point hash functions. To enhance the anonymity of user identities, the algorithm uses the Schnorr identity authentication protocol for identity authentication. Collusion attack is a type of attack in which multiple malicious entities collaborate or share information to bypass security mechanisms [36]. Xiong et al. [37] proposed a certificateless aggregate signature scheme for conditional privacy-preserving batch authentication, which can resist collusion attacks and offers high verification efficiency. However, since the scheme does not introduce random values during the signature phase, any attacker can forge a valid signature by obtaining the user's secret value from two legitimate signatures. Therefore, Xiong et al.'s scheme is vulnerable to forgery attacks.

Table I provides a brief summary of the aforementioned related schemes and their existing problems. To address the issues of time-consuming bilinear pairing and hash-to-point operations, the vulnerability of pairings-free CLSs to Type-I and Type-II adversaries due to improper parameter selection, and the susceptibility of aggregate signatures to collusion attacks, this article proposes an efficient certificateless aggregate signature scheme. The scheme combines blockchain and smart contract technologies and is applied in smart healthcare, achieving multiparty secure authentication in medical scenarios.

III. Preliminaries

This section introduces the preparatory knowledge. We begin with the main building blocks of the system we propose, including certificateless signatures, hardness assumptions, blockchain, smart contracts, and security requirements, followed by the security model and system model.

A. Certificateless Signature Schemes

Certificateless signature is a type of public-key cryptographic digital signature scheme [23]. CLS schemes eliminate

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 18,2026 at 09:20:19 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON

Scheme	Paring-free	Type-I attack	Type-II attack	Batch-Verification	collusion attacks	Hardness Problems	Security Requirements
Shim [24]	×	×	√	×	×	CDH	×
Wang [19]	×	×	×	×	×	ECDLP	×
Wu [21]	×	√	√	√	√	co-CDH	×
Wang [26]	×	×	×	√	×	CDH	√
Jia [29]	√	×	×	×	×	ECDLP	×
Du [30]	√	×	√	×	×	ECDLP	×
Xiang [31]	√	×	√	×	×	EDCLP	×
Zhan [15]	√	×	√	√	×	EDCLP	×
Li [34]	√	√	√	√	×	EDCLP	×
Xiong [37]	√	×	×	√	√	EDCLP	√

Note: √ means able to resist the attack or implement the function, × means unable to resist the attack or implement the function. The security requirements include conditional privacy preservation, traceability, and resistance to replay attacks.

the need for certificate management, reducing overhead, and are generally composed of the following algorithms.

Setup: The algorithm is executed by the KGC. It takes a security parameter λ as input and outputs the system's public parameters *params* and the system's master key s. The public parameters *params* are made public, while the system's master key s is kept secret.

Extract-Partial-Private-Key: The algorithm is executed by the KGC. It takes the system's public parameters *params*, the system's master key s, and the user's identity ID as input, and outputs the user's partial private key k.

Set-Secret-Value: The algorithm is executed by the user. It takes the system's public parameters *params* and the user's identity ID as input, and outputs the user's secret value x.

Set-Public-Key: The algorithm is executed by the user. It takes the system's public parameters *params* and the user's secret value x as input, and outputs the user's public key PK.

Sign: The algorithm takes the system's public parameters *params*, the message m, the signer's identity ID, the partial private key k, and the user's secret value x as input, and outputs the signature σ for the message m.

Verify: The algorithm takes the system's public parameters *params*, the signature σ, the message m, the signer's identity ID, and the user's public key PK as input, and outputs the verification result. If the verification is successful, it outputs 1, otherwise, it outputs 0.

B. Hardness Problems

The security of the CLS in this article is based on the elliptic curve discrete logarithm problem (ECDLP) [38], which is described as follows.

ECDLP: Let $\mathbb{G}$ be an elliptic curve cyclic group defined over a finite field, with p being the order of $\mathbb{G}$, a large prime number. Let P be a generator of $\mathbb{G}$, and E be a random point on the elliptic curve. If there exists a relation $E = eP$, then it is computationally hard to determine e within probabilistic polynomial time (PPT). The ECDLP states that if there exists any PPT algorithm to solve the ECDLP, the probability ϵ is negligible, which is formulated as

$$\Pr[\mathcal{A}(E,eP) = e] \leq \text{negl}(\lambda)$$

where $\text{negl}(\lambda)$ is a negligible value with respect to the security parameter λ.

C. Blockchain

Blockchain is a distributed ledger technology [39] that allows multiple participants to collaboratively maintain a growing list of data records, known as blocks, which are linked together to form a blockchain. Due to its features such as decentralization, immutability, transparency, and traceability, blockchain is widely used in scenarios like identity verification, financial services, and the Internet of Things. Blockchain is typically combined with cryptography to ensure the security of the information recorded on the blockchain. Currently, the main types of blockchain include public chains, private chains, consortium chains, and sidechains.

D. Smart Contracts

A smart contract is a program that runs on a blockchain and automatically executes the terms of a contract when predefined conditions are met. The concept of smart contracts was first introduced by Szabo [40], who defined them as a digital transaction protocol designed to convey, verify, or enforce the terms of a contract in an automated manner. Smart contracts are deployed on a blockchain, and they possess characteristics such as public accessibility, permanence, and immutability.

E. Security Requirements

1) Identity Authentication: In smart healthcare systems, only authenticated doctors, patients, or devices are permitted to access the system, in order to prevent unauthorized access, data leakage or tampering, identity impersonation, and third-party interception. Upon initiating access, users must send an access request message, which is then verified by the system to ensure that the requester is a legitimate and authorized entity.

2) Nonrepudiation: In smart healthcare systems, ensuring who accessed the system and what operations were performed on medical data is essential for clarifying accountability, preventing medical disputes, and enabling trustworthy data exchange. Therefore, users should not be able to deny having accessed the system or having performed the corresponding actions.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 18,2026 at 09:20:19 UTC from IEEE Xplore. Restrictions apply.

3) Resistance to Replay Attacks: In smart healthcare systems, if authentication messages can be replayed, an attacker may impersonate a doctor or patient by simply replaying a previously valid login request—without needing to obtain any secret keys. Therefore, a secure system should possess the capability to discern between two identical messages, thereby preventing an attacker from exploiting the same message to gain authentication.

4) Traceability: In smart healthcare systems, malicious attackers may perform unauthorized or harmful operations. Therefore, it is critical for the system to promptly identify the source of such actions. Traceability enables the system to record and associate all operations with their initiators, ensuring that even if a malicious action is successfully executed, its origin can be investigated. This allows for full-cycle verifiability and controllability of medical activities. Accordingly, the system should be equipped with the capability to pinpoint the specific user after the execution of any illicit operations, thereby enabling the tracking of the user's activities.

5) Conditional Privacy Protection: The system must protect the identity and behavioral privacy of patients and medical professionals. During any operation, only the minimal necessary information should be disclosed to prevent privacy leakage and potential malicious exploitation. The true identity of the user remains anonymous during the communication process. Except for the system, which has the capability to reveal the user's system identity, neither other users nor potential attackers can ascertain the user's real identity.

6) Unlinkability: Legitimate users in the system frequently access medical data for viewing or modification purposes. If these operations can be linked to the same individual, then even without directly revealing the user's identity, an attacker may reconstruct behavioral patterns, infer health conditions, or even generate detailed identity profiles. Therefore, a secure system must ensure that an attacker is unable to correlate whether two or more messages originate from the same user.

7) Resistance to Collusion Attacks: In smart healthcare systems, multiple internal operators or users within a hospital may collude to bypass access control mechanisms and perform unauthorized operations. Therefore, a secure medical system must ensure that multiple attackers cannot collaborate to deceive the system and thereby succeed in passing verification.

F. Security Model

As stated in [22], [23], and [34], Type-I and Type-II attacks are two types of attacks that may threaten the security of CLS protocols, an unforgeable signature should be resistant to both Type-I and Type-II attacks. The attack capability of a Type-I adversary $\mathcal{A}_1$ is that, although it cannot obtain the partial private keys of other users from the KGC, it can replace the public keys of other users. $\mathcal{A}_1$ simulates a malicious user. The attack capability of a Type-II adversary $\mathcal{A}_2$ is that it possesses the system's master key and the partial private key of the target user but cannot replace the target user's public key. $\mathcal{A}_2$ simulates a malicious but passive KGC.

The security of the CLS in this article will be analyzed by defining two games, Game-I and Game-II, to counter Type-I and Type-II attacks. The attackers $\mathcal{A}_1$ and $\mathcal{A}_2$ are allowed to perform the following five types of queries.

1) *User Creation Query*: $\mathcal{A}$ provides a pseudonymous identity ID_R, and when a public key request for the pseudonymous identity ID_R is received, C queries the list L_Q and then responds with the public key PK of the pseudonymous identity ID_R.
2) *Partial Private Key Query*: $\mathcal{A}$ provides a pseudonymous identity ID_R, and when a partial private key request for the pseudonymous identity ID_R is received, C queries the list L_Q and then responds with the partial private key k of the pseudonymous identity ID_R.
3) *User Secret Value Query*: $\mathcal{A}$ provides a pseudonymous identity ID_R, and when a secret value request for the pseudonymous identity ID_R is received, C queries the list L_Q and then responds with the secret value x of the pseudonymous identity ID_R.
4) *Public Key Replacement Query*: $\mathcal{A}$ provides a pseudonymous identity ID_R and a replacement public key $PK^* = Q^*$. C queries the list L_Q and replaces the public key PK of the pseudonymous identity ID_R with the new public key PK^*.
5) *Signature Query*: $\mathcal{A}$ provides a pseudonymous identity ID_R and a message m. When a signature request for m from the pseudonymous identity ID_R is received, C generates the corresponding signature σ and responds to $\mathcal{A}$.

Game-I: The game between the Type-I attacker $\mathcal{A}_1$ and the challenger C_1 is as follows.

1) The challenger C_1 performs system initialization, generating the system's master private key s, the system's master public key P_{pub}, and the system's public parameters *params*. The challenger C_1 sends the system's public parameters *params* to the attacker $\mathcal{A}_1$ and maintains the secrecy of the system's master private key s.
2) The attacker $\mathcal{A}_1$ executes the following oracle queries: user creation query, partial private key query, user secret value query, public key replacement query, and signature query.
3) The attacker $\mathcal{A}_1$ submits the target pseudonymous identity $ID_R{}^*$ and the forged signature σ^*.

If the output of the attacker $\mathcal{A}_1$ satisfies the following conditions, $\mathcal{A}_1$ wins the game.

1) σ^* is a valid signature of the pseudonymous identity $ID_R{}^*$ on the message m^*.
2) The attacker $\mathcal{A}_1$ has not used the $(ID_R{}^*, m^*)$ pair to execute a signature query.
3) The attacker $\mathcal{A}_1$ has not submitted the $ID_R{}^*$ to the Partial private key query to obtain the partial private key.

Definition 1: If there exists an attacker $\mathcal{A}_1$ whose probability of winning the game in Game-I is negligible, then the scheme satisfies the first type of security.

Game-II: The game between the Type-II attacker $\mathcal{A}_2$ and the challenger C_2 is as follows.

1) The challenger C_2 performs system initialization, generating the system's master private key s, the system's master public key P_{pub}, and the system's public parameters *params*. The challenger then sends the system's

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 18,2026 at 09:20:19 UTC from IEEE Xplore. Restrictions apply.

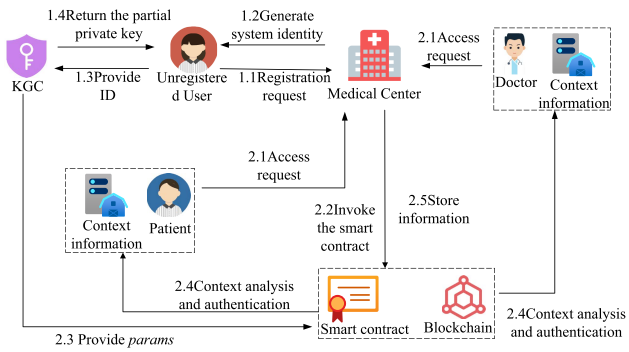

Fig. 1. System model.

public parameters *params* and the system's master private key s to the attacker.

2) The attacker $\mathcal{A}_2$ executes the following oracle queries: user creation query, partial private key query, user secret value query, and signature query.
3) The attacker $\mathcal{A}_2$ submits the target pseudonymous identity $ID_R{}^*$ and the forged signature σ^*.

If the output of the attacker $\mathcal{A}_2$ satisfies the following conditions, $\mathcal{A}_2$ wins the game.

1) σ^* is a valid signature of the pseudonymous identity $ID_R{}^*$ on the message m^*.
2) The attacker $\mathcal{A}_2$ has not used the $(ID_R{}^*, m^*)$ pair to execute a signature query.
3) The attacker $\mathcal{A}_2$ has not submitted the $ID_R{}^*$ to the user secret value query to obtain the user's secret value.

Definition 2: If there exists an attacker $\mathcal{A}_2$ whose probability of winning the game in Game-II is negligible, then the scheme satisfies the second type of security.

G. System Model

A blockchain-based medical data sharing model is proposed to address the authentication problem in the data sharing domain. The model mainly includes four types of entities, such as user, medical center (MC), KGC, and blockchain, as shown in Fig. 1. An unregistered user sends a registration request to the MC, which generates a system identity ID_{sys} for the user, both the MC and users need to store the user's system identity. The user then uses the system identity ID_{sys} to generate the corresponding pseudonym ID and submits it to the KGC, which returns part of the private key to the user, the user needs to securely store part of the private key. A registered user then sends an access request to the MC. Through relevant devices, the user's environmental context and biometric context are extracted and submitted to the MC. The MC invokes the smart contract on the blockchain and uses the public parameters generated by the KGC to determine the security of the user context and the legality of the user identity. The final results are given to the users, and the access information is recorded.

1) *User:* This includes roles such as doctors and patients who need to access the medical system. Unregistered doctors and patients must register with the medical system to obtain a system identity before they can perform operations such as access, and the user needs to store their system identity. Once registered, doctors and patients become legitimate users of the system. The user needs to request a partial private key from the KGC and securely store it. When users need to access the system, they are required to provide signatures and contextual information to the MC for authentication. User context information includes the user's environmental context and biometric context.
2) *MC:* It is mainly used to generate and store the user's system identity, as well as to accept the user's access requests and initiate relevant function call requests to the BC. It is used to predefine and store the secure environmental context $\{sc, w\}$. Additionally, it receives registration requests from unregistered users, generates and stores the user's system identity ID_{sys} and stores the processed biometric features *sbio* on the BC. It also receives access requests from registered users, verifies their identity by invoking smart contracts deployed on the blockchain, and stores the user's access information on the blockchain.
3) *KGC:* It is mainly used to generate system parameters and the user's partial private key. It involves generating system parameters and the master private key, making the system parameters public, and securely storing the master private key. It receives the user's ID and generates the corresponding partial private key, which will be used for identity authentication during subsequent user access. After the user registers with the medical system and obtains the identity ID_{sys}, they use ID_{sys} to generate an anonymous identity ID_R and provide it to the KGC. The KGC then generates the corresponding partial private key for the user and securely returns it to the user through a secure channel.
4) *Blockchain (BC):* It mainly consists of the blockchain and the smart contracts deployed on the blockchain. It is mainly used to execute relevant smart contracts and store information. When the MC initiates a request for the relevant function call, it performs context information analysis and verify the user's signature through smart contracts, assessing the security of the user's contextual information and authenticating the user's identity. Additionally, it stores user access records and biometric features *sbio*.

IV. Our Proposed Scheme

In this section, we mainly introduce our specific scheme. The specific scheme consists of three phases: 1) the user registration phase; 2) the initialization phase; and 3) the authentication phase. The relevant symbol definitions are shown in Table II. The first phase is the user registration phase, where the user submits a registration request to the medical system and provides their legitimate information to obtain a system identity. The second phase is the initialization phase, which primarily involves the initialization of the system and user key generation. The third phase is the user authentication phase, where the user provides their signature for identity authentication. Upon receiving the message, The smart contract utilizes a secure environmental context and a biometric similarity function to determine whether the user's contextual

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 18,2026 at 09:20:19 UTC from IEEE Xplore. Restrictions apply.

TABLE II
SYMBOL DEFINITIONS IN THIS ARTICLE

Symbol	Description
s	The system's (KGC) master private key
P_{pub}	The system's (KGC) master public key
ID_{sys}	The user's system identity
ID_R	The pseudonymous identity generated by the user
x	The user's secret value
k	The user's partial private key
$H_{i(i=0,1,2)}$	Secure hash function
σ	Signature information
M	Complete signed message
$context$	User context information set
c_i	The extracted user environment
sc_i	The security environment of the system has been defined
$sim()$	The function to calculate the similarity of biometric features
bio_i	The extracted user biometric features
$sbio_i$	Biometric features
w	Security weight
W_{secur}	The pre-defined weight threshold of the system
$\lvert\cdot\rvert$	The length of the data

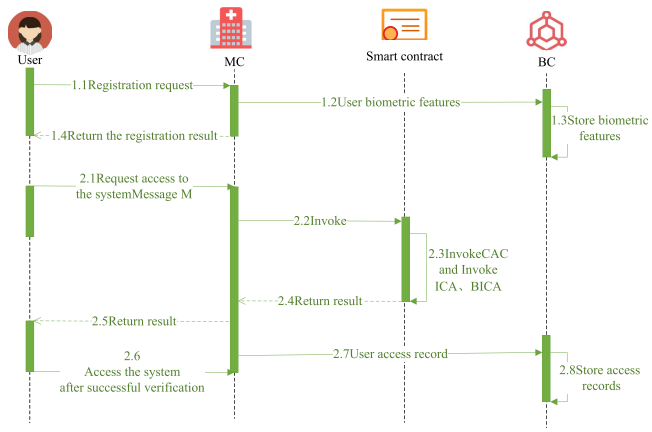

Fig. 2. Flowchart of the scheme.

environment is secure, while also verifying the legality of the signature. If valid, the authentication is successful; otherwise, the authentication fails. Biometric features and biometric similarity functions have been studied and applied [41]. In this article, we only utilize the related concepts to construct our scheme. The process of the scheme is shown in Fig. 2.

A. Registration Phase

Before using the services or data, users must register with the MC to obtain their system identity ID_{sys} within the system. This identity is used to bind the user, and users must authenticate their identity whenever logging into the system to use services. Users are required to provide their truthful and reliable information, including biometric features such as fingerprints and irises, as well as other identity-related information. The MC verifies the provided information to determine whether it is legitimately issued. If the information is valid, the MC generates a system identity for the user and stores the user's biometric features along with the encrypted user information on the blockchain, this information will subsequently be incorporated as part of the user's secure biometric context. Otherwise, the registration fails. If the registration fails, the user can submit another registration request to the system.

B. Initialization Phase

This phase includes System Initialization, User Pseudonymous Identity Generation, Partial Private Key Generation, and User key generation.

1) System Initialization: Let $\mathbb{G}$ be a defined elliptic curve over the finite field F_p, where p is a secure prime number, and P is the generator of $\mathbb{G}$. $\mathbb{Z}_q^*$ is the integer group of order q.

1) Choose three hash functions H_0, H_1, H_2 such that:
   $H_0 : \mathbb{G} \rightarrow \mathbb{Z}_q^*$,
   $H_1 : \{0, 1\}^* \times \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{Z}_q^*$,
   $H_2 : \{0, 1\}^* \times \{0, 1\}^* \times \mathbb{G} \times \mathbb{G} \times \mathbb{G} \times \{0, 1\}^* \rightarrow \mathbb{Z}_q^*$.
2) Randomly select the system's master private key $s \in \mathbb{Z}_q^*$, and calculate $P_{\text{pub}} = sP$, P_{pub} is the system's master public key.
3) Store the predefined security context environment values $\{sc_i, w_i\}$ locally and securely, where sc_i represents a predefined security environment context, and w_i denotes the corresponding weight.
4) Upload the public parameter $params = \{P, q, H_0, H_1, H_2, P_{\text{pub}}\}$ to the blockchain for storage.

2) User Pseudonymous Identity Generation: The user randomly selects $y \in \mathbb{Z}_q^*$, then computes

$$Y = yP$$
$$ID_R = ID_{sys} \oplus H_0\left(yP_{\text{pub}}\right).$$

The pseudonymous identity is obtained as $\mathrm{ID} = \{\mathrm{ID}_R, Y\}$.

3) Partial Private Key Generation: The user obtains a partial private key through the following process.

1) The user selects $x \in \mathbb{Z}_q^*$ as the secret value, computes $X = xP$, and sends $\{\mathrm{ID}, X\}$ to the KGC.
2) Upon receiving the message, the KGC first calculates $\mathrm{ID}_{sys} = \mathrm{ID}_R \oplus H_0(Ys)$ to obtain the user's system identity.
3) KGC checks whether the user's system identity is valid. If valid, the KGC generates the user's partial private key k using the user's pseudonymous identity ID_R. KGC chooses a random number $r \in \mathbb{Z}_q^*$ computes

$$R = rP$$
$$\alpha = H_1\left(ID_R, R + X, P_{\text{pub}}\right)$$
$$k = r + s\alpha \bmod q.$$

Correctness: The reasons why the KGC can restore the user's system identity are as follows:

$$\begin{aligned} ID_{sys} &= ID_R \oplus H_0(Ys) \\ &= ID_R \oplus H_0(ysP) \\ &= ID_R \oplus H_0\left(yP_{\text{pub}}\right) \end{aligned}$$

4) User Key Generation: After the user receives the message from the KGC.

1) The user first computes $Q = X + R$ and then verifies whether $kP = R + H_1(\mathrm{ID}_R, Q, P_{\text{pub}})P_{\text{pub}}$ holds.
2) If it holds, the user accepts this partial private key.
3) Otherwise, the user can resend $\{\mathrm{ID}, X\}$ to the KGC to obtain a new partial private key.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 18,2026 at 09:20:19 UTC from IEEE Xplore. Restrictions apply.

4) The user obtains the private key $SK = x$, and the user's public key is $PK = Q$.

Correctness: The verification of the partial private key k is as follows:

$$\begin{aligned} kP &= rP + s\alpha P \\ &= R + \alpha P_{\text{pub}} \\ &= R + H_1\big(ID_R, Q, P_{\text{pub}}\big)P_{\text{pub}}. \end{aligned}$$

C. Authentication Phase

This phase includes User context collection, User signature, Identity authentication, Aggregate signature, and Batch verification.

1) User Context Collection: When the user needs to access the system, the user context environment is first extracted through context-related devices to obtain the user context information context $= \{c_1, c_2, \ldots, c_n, \text{bio}_1, \text{bio}_2, \ldots, \text{bio}_m\}$, which includes user environmental context such as time, location, etc., as well as biometric context such as fingerprints, iris patterns, and other biometric features.

2) User Signature: A corresponding signature needs to be generated for identity authentication.

1) The user randomly selects a message m, and a random number $u \in \mathbb{Z}_q^*$, then calculates

$$\begin{aligned} U &= uX \\ \beta &= H_2\big(ID_R, m, Q, U, P_{\text{pub}}, T\big) \\ v &= ux\beta + x + k \bmod q \end{aligned}$$

where T is the timestamp.

2) The signature information $\sigma = \{U, v\}$ is generated, and the user sends the message $M = \{m, PK, \sigma, \text{ID}, T, \text{context}\}$ through a secure channel to the MC for verification.

3) Identity Authentication: After MC receives the message M.

1) The MC determines the freshness of the message using the timestamp T, and obtains the user's context information $\{c_1, c_2, \ldots, c_n, \text{bio}_1, \text{bio}_2, \ldots, \text{bio}_m\}$ through context.
2) The smart contract calls the context analysis function *CAC* as shown in Algorithm 1, taking the user's context information as input.
3) If there exists $c_i \in sc$, i.e., there exists $c_i = sc_j$, the corresponding security weight w_i for sc_j is obtained.
4) If for any sc_j, $c_i \notin sc$, then $w_i = 0$.
5) The formula for the user context security value w is

$$w = \sum_{i=1}^{n} w_i.$$

6) Using the biometric similarity function *sim*() and the user's reserved biometric $sbio_i$, calculate the user's biometric security value d_i

$$\begin{aligned} d_i &= sim(\text{bio}_i, sbio_i) \\ d &= \sum_{i=1}^{m} d_i. \end{aligned}$$

7) The total weight $W = w + d$ is calculated and compared with the system's security threshold W_{secur}. If $W < W_{\text{secur}}$, access is denied; otherwise, identity authentication proceeds. The MC calls the smart contract Algorithm 2 *IAC* or Algorithm 3 *BIAC* function to authenticate the user's identity, calculating

$$\begin{aligned} \alpha &= H_1\big(ID_R, Q, P_{\text{pub}}\big) \\ \beta &= H_2\big(ID_R, m, Q, U, P_{\text{pub}}, T\big) \end{aligned}$$

and verifying if $vP = \beta U + Q + \alpha P_{\text{pub}}$ holds. If it does not hold, identity authentication fails; otherwise, identity authentication succeeds, and the user can successfully access the system.

Algorithm 1: Context Analysis Contract (CAC)

Input:User's context information *context*, system-stored security environment *CONTEXT*, system security threshold W_{secur}
Output:result_con

```
w=0
for i=1:n do
    if c_i∈sc then // If the context information exists
        w=w+w_i // Obtain the corresponding weights and
                // calculate the sum
    else then
        w=w+0
    end
end
end
for i=1:n do
    d_i=sim(bio_i,sbio_i) // Directly obtain the weights through
                          // the sim() function
    w=w+d_i
end
if w < W_secur then
    result_con=0 // If it is smaller than the security threshold,
                 // access is denied
end
else
    result_con=1 // If it is greater than the security threshold,
                 // perform identity authentication.
end
return result_con
```

Correctness: The verification of the signature σ can be extended as follows:

$$\begin{aligned} vP &= ux\beta P + xP + kP \\ &= u\beta X + X + R + H_1\big(ID_R, Q, P_{\text{pub}}\big)P_{\text{pub}} \\ &= \beta U + Q + H_1\big(ID_R, Q, P_{\text{pub}}\big)P_{\text{pub}} \\ &= \beta U + Q + \alpha P_{\text{pub}}. \end{aligned}$$

4) Aggregate Signature: When multiple messages $\{M_1, M_2, \ldots, M_n\}$ are obtained, the MC randomly selects $L = \{\text{l}_1, \text{l}_2, \ldots, \text{l}_n\}$, and computes

$$\begin{aligned} \mathcal{V} &= \sum_{i=1}^{n} l_i v_i, \\ \mathcal{Q} &= \sum_{i=1}^{n} l_i Q_i, \end{aligned}$$

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 18,2026 at 09:20:19 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: Identity Authentication Contract (IAC)

Input:User identity information *ID*, user public key *PK*, signature information σ, and system public parameters *params*, message *m*
Output:result_verify
1.α=$H_1(ID_R, Q, P_{pub})$
2.β=$H_2(ID_R, m, Q, U, P_{pub}, T)$
3.**if** vP==βU+Q+αP_{pub} **then**
4. result_verify=1 // Identity authentication passed
5.**end**
6.**else**
7. result_verify=0 // Identity authentication failed
8.**end**
9.**return** result_verify

Algorithm 3: Batch Identity Authentication Contract (BIAC)

Input:Aggregate message $\{\mathcal{V}, L, \mathcal{Q}, \mathcal{N}\}$, messages from n users $\{m_i, PK_i, ID_i, T_i, CONTEXT_i\}$, and system public parameters *params*
Output:result_bverify
1.β_i=$H_2(ID_{R,i}, m_i, Q_i, U_i, P_{pub}, T_i)$
2.**if** $\mathcal{V}P$==$\sum_{i=1}^{n} l_i\beta_i U_i + \mathcal{Q} + \mathcal{N}P_{pub}$ **then**
3. result_bverify=1 // Identity authentication passed
4.**end**
5.**else**
6. result_bverify=0 // Identity authentication failed
7.**end** 8.**return** result_bverify

$$\alpha_i = H_1\big(ID_{R,i}, Q_i, P_{\text{pub}}\big),$$
$$\mathcal{N} = \sum_{i=1}^{n} l_i\alpha_i.$$

The aggregated message $\{\mathcal{V}, L, \mathcal{Q}, \mathcal{N}\}$ is then output.

5) Batch Verification: When n messages $\{m_i, PK_i, \text{ID}_i, T_i, CONTEXT_i\}$ and the aggregated message $\{\mathcal{V}, L, \mathcal{Q}, \mathcal{N}\}$ are obtained.

1) The MC first checks the freshness of each message.
2) If the freshness check passes, it then verifies whether the context information of each message is secure.
3) If the context is secure, it proceeds to calculate $\beta_i = H_2(\text{ID}_{R,i}, m_i, Q_i, U_i, P_{\text{pub}}, T_i)$ and compute

$$\mathcal{V}P = \sum_{i=1}^{n} l_i\beta_i U_i + \mathcal{Q} + \mathcal{N}P_{\text{pub}}$$

check if the equation holds. If it holds, the identity authentication for these n messages is successful; otherwise, the authentication fails.

Correctness: The verification of the signature $\{\mathcal{V}, L, \mathcal{Q}, \mathcal{N}\}$ can be extended as follows:

$$\begin{aligned}
\mathcal{V}P &= \sum_{i=1}^{n} l_i v_i P \\
&= \sum_{i=1}^{n} l_i(u_i x_i \beta_i + x_i + k_i)P \\
&= \sum_{i=1}^{n} l_i u_i x_i \beta_i P + \sum_{i=1}^{n} l_i x_i P + \sum_{i=1}^{n} l_i k_i P \\
&= \sum_{i=1}^{n} l_i u_i x_i \beta_i P + \sum_{i=1}^{n} l_i X_i + \sum_{i=1}^{n} l_i(R_i + \alpha_i sP) \\
&= \sum_{i=1}^{n} l_i \beta_i U_i + \sum_{i=1}^{n} l_i X_i + \sum_{i=1}^{n} l_i R_i + \sum_{i=1}^{n} l_i \alpha_i sP \\
&= \sum_{i=1}^{n} l_i \beta_i U_i + \sum_{i=1}^{n} l_i (X_i + R_i) + \mathcal{N}P_{\text{pub}} \\
&= \sum_{i=1}^{n} l_i \beta_i U_i + \sum_{i=1}^{n} l_i Q_i + \mathcal{N}P_{\text{pub}} \\
&= \sum_{i=1}^{n} l_i \beta_i U_i + \mathcal{Q} + \mathcal{N}P_{\text{pub}}.
\end{aligned}$$

V. Security Analysis

In this section, we will prove that our scheme satisfies security requirements such as identity authentication, nonrepudiation, resistance to replay attacks, traceability, conditional privacy protection, unlinkability, and resistance to collusion attacks through informal security analysis. Subsequently, we will demonstrate the security of our scheme through formal security analysis, showing that it can resist Type-I and Type-II adversaries, thus proving the security of our scheme.

A. Informal Security Analysis

1) Identity Authentication: Every time a user attempts to access the system, they must generate a corresponding signature. Upon receiving the message, the system performs identity authentication using the user's public key and the system's public parameters. The message's authenticity is verified by computing $vP = \beta U + Q + \alpha P_{\text{pub}}$, to ensure that the message indeed comes from the sender associated with the current verification public key.

2) Nonrepudiation: In the scheme, when signing information, the user not only uses part of their private key but also the secret value they employ. Additionally, the anonymous identity obtained by the user through the system's identity is included in the hashing process during signing. As a result, the user cannot deny having signed the message, ensuring the scheme's nonrepudiation.

3) Resistance to Replay Attacks: In the scheme, a random number is chosen during the signing process, and a timestamp is also introduced. As a result, the signature generated by the user for different messages is unique. Even if the same message is signed, the signature will differ due to the varying timestamps. If the medical system receives a message identical to a previous one, access will be denied, thereby preventing replay attacks.

4) Traceability: In the scheme, when a user registers with the system, their encrypted information is stored on the blockchain along with the system-assigned identity ID_{sys}. If the user engages in any illicit activity, the blockchain can interact with the ID_R and KGC to restore the user's system identity ID_{sys}. Additionally, the blockchain can locate the

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 18,2026 at 09:20:19 UTC from IEEE Xplore. Restrictions apply.

corresponding encrypted information using the ID_{sys}. Once decrypted, the original user information can be obtained, enabling user traceability.

5) Conditional Privacy Protection: In the scheme, during the signing process, the user-generated pseudonym ID_R is used for computation. The signed message contains no information related to the user. Moreover, since the generation of the pseudonym is based on $\mathrm{ID}_R = \mathrm{ID}_{sys} \oplus H_0(yP_{\mathrm{pub}})$, to obtain the user's system identity, one would need to compute $\mathrm{ID}_{sys} = \mathrm{ID}_R \oplus H_0(Ys)$. As s is the system's master private key, attackers cannot compute the user's system identity. Therefore, the scheme ensures conditional privacy protection.

6) Unlinkability: In the scheme, an attacker cannot determine whether any two signed messages come from the same user, because the user selects a random number during signing and uses pseudonymous information instead of real identity information. As a result, the attacker is unable to link any two messages together and determine if they originated from the same user. Therefore, the scheme ensures unlinkability.

7) Resistance to Collusion Attacks: For aggregated signatures that do not introduce a random number, two malicious users can collaborate on a random value $\mathrm{e} \in \mathbb{Z}_q^*$ to generate two illegal signatures $v_i' = v_i + \mathrm{e}$ and $v_j' = v_j - \mathrm{e}$. Since the aggregated signature $\mathcal{V} = \sum_{k=1, k\neq i, k\neq j}^{n} v_k + v_i' + v_j' = \sum_{k=1} v_k$, this aggregated signature can satisfy $\mathcal{V}P = \sum_{i=1}^{n} \beta_i U_i + \sum_{i=1}^{n} Q_i + \sum_{i=1}^{n} \alpha_i$ and pass batch verification. Additionally, two malicious users can exchange part of their private keys to form two illegal signatures $v_i = u_i x_i \beta_i + k_j + x_i, v_j = u_j x_j \beta_j + k_i + x_j$.Similarly, the aggregated signature of these two signatures can pass batch verification. However, since the scheme selects a random number for each signature during the aggregation process, $L_i \mathrm{e} \neq L_j \mathrm{e}, L_i R_j + L_i X_i \neq L_i Q_i$, thus preventing successful batch verification. Therefore, the scheme can resist collusion attacks.

B. Formal Security Analysis

In a certificateless cryptosystem, $\mathcal{A}_1$ and $\mathcal{A}_2$ are two adversaries with different attack capabilities. Assume that $\mathcal{A}_1$ models a dishonest user, while $\mathcal{A}_2$ is a malicious but passive KGC.

1) *Type-I Adversary:* $\mathcal{A}_1$ does not know the system's master key or the user's partial private key, but can substitute the user's public key.
2) *Type-II Adversary:* $\mathcal{A}_2$ knows the system's master key and the user's partial private key, but cannot substitute the user's public key.

Theorem 2: In the random oracle model, if the adversary $\mathcal{A}_1$ can successfully forge a signature with a nonnegligible probability, then the challenger C_1 can solve the ECDLP problem with a nonnegligible probability. Therefore, the scheme is considered to have unforgeability under the adaptive chosen-message and identity attack of adversary $\mathcal{A}_1$.

Proof: The challenger C_1 aims to solve the ECDLP problem by inputting $(P_{\mathrm{pub}} = sP, P)$, with the goal of obtaining the secret key s through interaction with the adversary $\mathcal{A}_1$ in order to forge a valid signature, where s is the solution to the ECDLP problem.

System Initialization Phase: C_1 executes the system initialization algorithm. C_1 randomly selects $s \in \mathbb{Z}_q^*$, computes $P_{\mathrm{pub}} = sP$, and sends the system public parameters $\{p, q, H_0, H_1, H_2, P_{\mathrm{pub}}\}$ to $\mathcal{A}_1$, while secretly storing s, Additionally, C_1 initializes and maintains the lists $L_{H_0} = \{(yP_{\mathrm{pub}}, h_0)\}$, $L_Q = \{(\mathrm{ID}_R, k, x, Q, X, P_{\mathrm{pub}}, h_1)\}$, $L_{H_2} = \{(\mathrm{ID}_R, m, Q, U, P_{\mathrm{pub}}, T, h_2)\}$, $L_{ID} = \{(\mathrm{ID}_R, y)\}$, and $L_v = \{(\mathrm{ID}_R, v, U, m)\}$.

Query Phase: $\mathcal{A}_1$ can perform the following queries.

1) H_0 Query: $\mathcal{A}_1$ sends a query request for (yP_{pub}). C_1 checks the list L_{H_0}. If (yP_{pub}, h_0) exists in L_{H_0}, C_1 returns $h_0 = H_0(yP_{\mathrm{pub}})$ to $\mathcal{A}_1$. If (yP_{pub}) does not exist in L_{H_0}, C_1 randomly selects $h_0 \in \mathbb{Z}_q^*$, adds the element (yP_{pub}, h_0) to the list L_{H_0} and finally returns the new element to $\mathcal{A}_1$.

2) User Creation Query: $\mathcal{A}_1$ sends a query request for $(\mathrm{ID}_R, k, x, Q, X, P_{\mathrm{pub}}, h_1)$. C_1 checks the list L_Q. If $\mathrm{ID}_R \neq \mathrm{ID}_R{}^*$, C_1 randomly selects k, x, and $h_1 \in \mathbb{Z}_q^*$, calculates $X = xP, Q = X + kP - P_{\mathrm{pub}} h_1$, and sets $h_1 = H_1(\mathrm{ID}_R, R + X = Q, P_{\mathrm{pub}})$, C_1 then adds $(\mathrm{ID}_R, k, x, Q = R + X, X, P_{\mathrm{pub}}, h_1)$ to the list L_Q and returns $PK = Q$ to the user. If $\mathrm{ID}_R = \mathrm{ID}_R{}^*$, C_1 randomly selects x, r, and $h_1 \in \mathbb{Z}_q^*$, calculates $X = xP$, $R = rP$, $k = r + sH_1(\mathrm{ID}_R, R + X, P_{\mathrm{pub}})$, $Q = R + X$, and $h_1 = H_1(\mathrm{ID}_R, R + X, P_{\mathrm{pub}})$. C_1 then adds $(\mathrm{ID}_R, k, Q = R + X, X, x, h_1)$ to the list L_Q and returns $PK = Q$ to $\mathcal{A}_1$.

3) H_1 Query: $\mathcal{A}_1$ sends a query request for $(\mathrm{ID}_R, Q = R + X, P_{\mathrm{pub}})$. C_1 checks the list L_Q, If an element $(\mathrm{ID}_R, Q, P_{\mathrm{pub}}, h_1)$ already exists in L_Q, C_1 returns $h_1 = H_1(\mathrm{ID}_R, Q, P_{\mathrm{pub}})$ to $\mathcal{A}_1$. Otherwise, C_1 initiates a User Creation Query and returns h_1 to $\mathcal{A}_1$.

4) H_2 Query: $\mathcal{A}_1$ sends a query request for $(\mathrm{ID}_R, m, Q, U, P_{\mathrm{pub}}, T)$. C_1 checks the list L_{H_2}. If an element $(\mathrm{ID}_R, m, Q, U, P_{\mathrm{pub}}, T, h_2)$ already exists in L_{H_2}, C_1 returns $h_2 = H_2(\mathrm{ID}_R, m, Q, U, P_{\mathrm{pub}}, T)$ to $\mathcal{A}_1$. Otherwise, C_1 randomly selects $h_2 \in \mathbb{Z}_q^*$, sets $h_2 = H_2(\mathrm{ID}_R, m, Q, U, P_{\mathrm{pub}}, T)$, adds the element $(\mathrm{ID}_R, m, Q, U, P_{\mathrm{pub}}, T, h_2)$ to the list L_{H_2}, and returns the new element to $\mathcal{A}_1$.

5) Anonymity Query: $\mathcal{A}_1$ sends a query for the pseudonymous identity ID_R. C_1 checks the list L_{ID}. If an element (ID_R, y) already exists in L_{ID}, C_1 directly returns y to $\mathcal{A}_1$. Otherwise, C_1 randomly selects $y \in \mathbb{Z}_q^*$, adds the element (ID_R, y) to L_{ID}, and returns the new element to $\mathcal{A}_1$.

6) Partial Private Key Query: $\mathcal{A}_1$ queries the partial private key for the pseudonymous identity ID_R. C_1 checks the list L_Q. If $\mathrm{ID}_R = \mathrm{ID}_R{}^*$, C_1 aborts. Otherwise, if an element $(\mathrm{ID}_R, k, x, Q, X, P_{\mathrm{pub}}, h_1)$ already exists in L_Q, C_1 returns the parameter to $\mathcal{A}_1$. Otherwise, C_1 initiates a user creation query and finally returns k to $\mathcal{A}_1$.

7) User Secret Value Query: $\mathcal{A}_1$ queries the secret value for the pseudonymous identity ID_R. C_1 checks the list L_Q. If an element $(\mathrm{ID}_R, k, x, Q, X, P_{\mathrm{pub}}, h_1)$ already exists in L_Q, C_1 returns the secret value x to $\mathcal{A}_1$. Otherwise, C_1 initiates a user creation query and returns x to $\mathcal{A}_1$.

8) Public Key Replacement Query: $\mathcal{A}_1$ initiates a public key replacement query for the pseudonymous identity ID_R. $\mathcal{A}_1$ forges a new public key $PK^* = Q^*$ and sends (ID_R, Q^*) to C_1. C_1 checks the list L_Q, If an element (ID_R, Q^*) already exists in L_Q, C_1 updates the corresponding parameter to $(\mathrm{ID}_R, \perp, \perp, Q^*, P_{\mathrm{pub}}, h_1)$.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 18,2026 at 09:20:19 UTC from IEEE Xplore. Restrictions apply.

9) Signature Query: $\mathcal{A}_1$ queries the signature for the pseudonymous identity ID_R and message m. If $\mathrm{ID}_R \neq \mathrm{ID}_R{}^*$ and Q has not been replaced, C_1 randomly selects u and $h_2 \in \mathbb{Z}_q^*$, calculates $U = uxP = uX$, sets $h_2 = H_2(\mathrm{ID}_R, m, Q, U, P_{\mathrm{pub}}, T)$, calculates $v = uxh_2+x+k$, and adds (ID_R, v, U, m) and $(\mathrm{ID}_R, m, Q, U, P_{\mathrm{pub}}, T)$ to L_v and L_{H_2}, respectively. Finally, C_1 returns $\sigma = \{U, v\}$ to $\mathcal{A}_1$. Otherwise, C_1 randomly selects v and $h_2 \in \mathbb{Z}_q^*$, calculates $U = vP\text{-}Q\text{-}h_2P_{\mathrm{pub}}$, sets $h_2 = H_2(\mathrm{ID}_R, m, Q, U, P_{\mathrm{pub}}, T)$, and returns $\sigma = \{U, v\}$ to $\mathcal{A}_1$. C_1 also adds (ID_R, v, U, m) and $(\mathrm{ID}_R, m, Q, U, P_{\mathrm{pub}}, T)$ to L_v and L_{H_2}, respectively.

Signature Forgery Phase: $\mathcal{A}_1$ can obtain the signature $\sigma = \{U, v\}$ for the message m in polynomial time. If $\mathrm{ID}_R \neq \mathrm{ID}_R{}^*$, C_1 terminates. Otherwise, C_1 repeats the above process but uses a different value for H_1, namely $h_1{}^* = H_1(\mathrm{ID}_R, Q, P_{\mathrm{pub}})$, with $h_1{}^* \neq h_1$. C_1 obtains another valid signature $\sigma^* = \{U, v^*\}$ for m. Based on the forking lemma, the KGC's private key can be derived. The specific derivation process is as follows: $vP\text{–}v^*P = (Uh_2+Q+P_{\mathrm{pub}}h_1)\text{–}(Uh_2+Q+P_{\mathrm{pub}}h_1{}^*) = P_{\mathrm{pub}}(h_1\text{–}h_1{}^*) = sP(h_1\text{–}h_1{}^*)$ Therefore, it can be deduced that $s = ([(v - v^*)]/[(h_1 - h_1^*)])$.

Based on the above game, we know that if the simulation does not abort during the process, then the challenger can solve the ECDLP problem. Let ξ_1 denotes the event that C_1 does not abort the game during the query phase, ξ_2 denote the event that C_1 does not abort the game during the signature forgery phase, ξ_3 denote the event that $\mathcal{A}_1$ can forge two valid signature, and the probability that $\mathcal{A}_1$ forgery a signature is ε_1. Then, we can obtain $\Pr[\xi_1]=(1\text{-}(1/[q_p + q_k + 1]))^{q_p+q_k}$, $\Pr[\xi_2]=(1/[q_p + q_k + 1])$, and $\Pr[\xi_3]\geq(1\text{-}(1/e))((\varepsilon_1/q_{h1}))$. Where q_p denotes the number of partial private key queries, q_k denotes the number of secret value queries, and q_{h1} denotes the number of H_1 hash function queries. e is the base of the natural logarithm. Therefore, the probability that the game does not abort is $\Pr[\xi_1\wedge\xi_2\wedge\xi_3]\geq(1\text{–}(1/[q_p + q_k + 1]))^{q_p+q_k}(1/[q_p + q_k + 1])(1\text{–}(1/e))((\varepsilon_1/q_{h1}))$. When $q_p + q_k$ is sufficiently large, $(1\text{–}(1/[q_p + q_k + 1]))^{q_p+q_k}$ approaches e^{-1}. Therefore, C_1 can solve the ECDLP problem with probability $\Pr[\xi_1\wedge\xi_2\wedge\xi_3]\geq(1\text{–}(1/e))((\varepsilon_1/[e(q_p + q_k + 1)q_{h1}]))$. However, this contradicts the assumption of ECDLP. That is to say, the probability of $\mathcal{A}_1$ forging a legal signature is negligible. Thus, under the random oracle model and the ECDLP assumption, the adaptive chosen message and identity attack by attacker $\mathcal{A}_1$ is unforgeable, which means the proposed scheme is secure.

Theorem 3: In the random oracle model, if the adversary $\mathcal{A}_2$ can successfully forge a signature with a nonnegligible probability, then the challenger C_2 can solve the ECDLP problem with a nonnegligible probability. Therefore, the scheme is considered to have unforgeability under the adaptive chosen-message and identity attack of adversary $\mathcal{A}_2$.

Proof: The challenger C_2 aims to solve the ECDLP problem by inputting $(X = xP, P)$, with the goal of obtaining the secret key x through interaction with the adversary $\mathcal{A}_2$ in order to forge a valid signature, where x is the solution to the ECDLP problem.

System Initialization Phase: C_2 executes the system initialization algorithm. C_2 randomly selects $s \in \mathbb{Z}_q^*$, computes $P_{\mathrm{pub}} = sP$, and sends the system public parameters $\{p, q, H_0, H_1, H_2, P_{\mathrm{pub}}\}$ to $\mathcal{A}_2$, while secretly storing s, Additionally, C_1 initializes and maintains the lists $L_{H_0} = \{(yP_{\mathrm{pub}}, h_0)\}$, $L_Q = \{(\mathrm{ID}_R, k, x, Q, X, P_{\mathrm{pub}}, h_1)\}$, $L_{H_2} = \{(\mathrm{ID}_R, m, Q, U, P_{\mathrm{pub}}, T, h_2)\}$, $L_{ID} = \{(\mathrm{ID}_R, y)\}$, and $L_v = \{(\mathrm{ID}_R, v, U, m)\}$.

Query Phase: $\mathcal{A}_2$ can perform the following queries.

1) H_0 Query: $\mathcal{A}_2$ sends a query request for (yP_{pub}). C_2 checks the list L_{H_0}. If (yP_{pub}, h_0) exists in L_{H_0}, C_2 returns $h_0 = H_0(yP_{\mathrm{pub}})$ to $\mathcal{A}_2$. If (yP_{pub}) does not exist in L_{H_0}, C_2 randomly selects $h_0 \in \mathbb{Z}_q^*$, adds the element (yP_{pub}, h_0) to the list L_{H_0} and finally returns the new element to $\mathcal{A}_2$.

2) User Creation Query: $\mathcal{A}_2$ sends a query request for $(\mathrm{ID}_R, k, x, Q, X, P_{\mathrm{pub}}, h_1)$. C_2 checks the list L_Q. If $\mathrm{ID}_R \neq \mathrm{ID}_R{}^*$, C_2 randomly selects k, x, and $h_1 \in \mathbb{Z}_q^*$, calculates $X = xP$, $Q = X+kP\text{–}P_{\mathrm{pub}}h_1$, and sets $h_1 = H_1(\mathrm{ID}_R, R+X = Q, P_{\mathrm{pub}})$, C_2 then adds $(\mathrm{ID}_R, k, x, Q = R+X, X, P_{\mathrm{pub}}, h_1)$ to the list L_Q and returns $PK = Q$ to the user. If $\mathrm{ID}_R = \mathrm{ID}_R{}^*$, C_2 randomly selects x, r, and $h_1 \in \mathbb{Z}_q^*$, calculates $X = xP$, $R = rP$, $k = r+sH_1(\mathrm{ID}_R, R+X, P_{\mathrm{pub}})$, $Q = R+X$, and $h_1 = H_1(\mathrm{ID}_R, R+X, P_{\mathrm{pub}})$. C_2 then adds $(\mathrm{ID}_R, k, Q = R+X, X, x, h_1)$ to the list L_Q and returns $PK = Q$ to $\mathcal{A}_2$.

3) H_1 Query: $\mathcal{A}_2$ sends a query request for $(\mathrm{ID}_R, Q = R+X, P_{\mathrm{pub}})$. C_2 checks the list L_Q, If an element $(\mathrm{ID}_R, Q, P_{\mathrm{pub}}, h_1)$ already exists in L_Q, C_2 returns $h_1 = H_1(\mathrm{ID}_R, Q, P_{\mathrm{pub}})$ to $\mathcal{A}_2$. Otherwise, C_2 initiates a User Creation Query and returns h_1 to $\mathcal{A}_2$.

4) H_2 Query: $\mathcal{A}_2$ sends a query request for $(\mathrm{ID}_R, m, Q, U, P_{\mathrm{pub}}, T)$. C_2 checks the list L_{H_2}. If an element $(\mathrm{ID}_R, m, Q, U, P_{\mathrm{pub}}, T, h_2)$ already exists in L_{H_2}, C_2 returns $h_2 = H_2(\mathrm{ID}_R, m, Q, U, P_{\mathrm{pub}}, T)$ to $\mathcal{A}_2$. Otherwise, C_2 randomly selects $h_2 \in \mathbb{Z}_q^*$, sets $h_2 = H_2(\mathrm{ID}_R, m, Q, U, P_{\mathrm{pub}}, T)$, adds the element $(\mathrm{ID}_R, m, Q, U, P_{\mathrm{pub}}, T, h_2)$ to the list L_{H_2}, and returns the new element to $\mathcal{A}_2$.

5) Anonymity Query: $\mathcal{A}_2$ sends a query for the pseudonymous identity ID_R. C_2 checks the list L_{ID}. If an element (ID_R, y) already exists in L_{ID}, C_2 directly returns y to $\mathcal{A}_2$. Otherwise, C_2 randomly selects $y \in \mathbb{Z}_q^*$, adds the element (ID_R, y) to L_{ID}, and returns the new element to $\mathcal{A}_2$.

6) Partial Private Key Query: $\mathcal{A}_2$ queries the partial private key for the pseudonymous identity ID_R. $\mathcal{A}_2$ sends a request for the KGC's secret value. C_2 checks the list L_Q. If $\mathrm{ID}_R = \mathrm{ID}_R{}^*$, C_2 aborts. Otherwise, if an element $(\mathrm{ID}_R, k, x, Q, X, P_{\mathrm{pub}}, h_1)$ already exists in L_Q, C_2 returns the parameter to $\mathcal{A}_2$. Otherwise, C_2 initiates a user creation query and finally returns k to $\mathcal{A}_2$.

7) User Secret Value Query: $\mathcal{A}_2$ queries the secret value for the pseudonymous identity ID_R. C_2 checks the list L_Q. If an element $(\mathrm{ID}_R, k, x, Q, X, P_{\mathrm{pub}}, h_1)$ already exists in L_Q, C_2 returns the secret value x to $\mathcal{A}_2$. Otherwise, C_2 initiates a user creation query and returns x to $\mathcal{A}_2$.

8) Signature Query: $\mathcal{A}_2$ queries the signature for the pseudonymous identity ID_R and message m. If $\mathrm{ID}_R \neq \mathrm{ID}_R{}^*$, C_2 randomly selects $u \in \mathbb{Z}_q^*$, calculates $U = uxP = uX$, calculates $v = uxh_2+x+k$, where h_2 can be obtained from the list L_{H_2}, adds (ID_R, v, U, m) to L_v, and finally returns $\sigma = \{U, v\}$ to $\mathcal{A}_2$. Otherwise, C_2 randomly selects v, $h_2 \in \mathbb{Z}_q^*$, calculates $U = vP\text{-}Q\text{-}h_2P_{\mathrm{pub}}$, sets $h_2 = H_2(\mathrm{ID}_R, m, Q, U, P_{\mathrm{pub}}, T)$, and

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 18,2026 at 09:20:19 UTC from IEEE Xplore. Restrictions apply.

TABLE III
FUNCTION COMPARISON

Scheme	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10
Shim [18]	√	×	×	√	√	√	×	×	×	×
Xiong [37]	√	√	√	√	√	×	√	√	√	×
Li [34]	√	√	√	√	√	√	×	√	√	×
Yang [22]	√	×	√	√	√	√	√	√	√	×
Ma [32]	√	×	√	√	√	√	×	×	√	×
Yang [33]	√	√	√	√	√	√	×	×	√	×
Our scheme	√	√	√	√	√	√	√	√	√	√

returns $\sigma = \{U, v\}$ to $\mathcal{A}_2$. C_2 also adds (ID_R, v, U, m) and (ID_R, m, Q, U, P_{pub}, T) to L_v and L_{H_2}, respectively.

Signature Forgery Phase: $\mathcal{A}_2$ can obtain the signature $\sigma = \{U, v\}$ in polynomial time and can choose a different hash function to forge another valid signature $\sigma^* = \{U, v^*\}$, where $v^* = uxh_2{}^*+x+k$. Based on the forking lemma and the equation $v = uxh_2+x+k$, the private key of the KGC can be derived. The specific derivation process is as follows: $vP\text{-}v^*P = (Uh_2+X+R+P_{\mathrm{pub}}h_1)\text{-}(Uh_2{}^*+X+R+P_{\mathrm{pub}}h_1) = Uh_2\text{-}Uh_2{}^*$ Therefore, it can be deduced that $x = ([(v - v^*)]/[u(h_2 - h_2^*)])$.

Based on the above game, we know that if the simulation does not abort during the process, then the challenger can solve the ECDLP problem. Let $\mathcal{F}_1$ denotes the event that C_2 does not abort the game during the query phase, $\mathcal{F}_2$ denote the event that C_2 does not abort the game during the signature forgery phase, $\mathcal{F}_3$ denote the event that $\mathcal{A}_2$ can forge two valid signature, and the probability that $\mathcal{A}_2$ forgery a signature is ε_2. Then, we can obtain $\Pr[\mathcal{F}_1]=(1\text{-}(1/[q_p + q_k + 1]))^{q_p+q_k}$, $\Pr[\mathcal{F}_2]=(1/[q_p + q_k + 1])$, and $\Pr[\mathcal{F}_3]\geq(1\text{-}(1/e))((\varepsilon_1/[q_{h2}]))$. Where q_{h2} denotes the number of H_2 hash function queries. Therefore, the probability that the game does not abort is $\Pr[\mathcal{F}_1\wedge\mathcal{F}_2\wedge\mathcal{F}_3]\geq(1\text{-}(1/[q_p + q_k + 1]))^{q_p+q_k}(1/[q_p + q_k + 1])(1\text{-}(1/e))((\varepsilon_2/[q_{h2}]))$. When $q_p + q_k$ is sufficiently large, $(1\text{-}(1/[q_p + q_k + 1]))^{q_p+q_k}$ approaches e^{-1}. Therefore, C_2 can solve the ECDLP problem with probability $\Pr[\mathcal{F}_1\wedge\mathcal{F}_2\wedge\mathcal{F}_3]\geq(1\text{-}(1/e))((\varepsilon_1/[e(q_p + q_k + 1)q_{h2}]))$. However, this contradicts the assumption of ECDLP. That is to say, the probability of $\mathcal{A}_2$ forging a legal signature is negligible. Thus, under the random oracle model and the ECDLP assumption, the adaptive chosen message and identity attack by attacker $\mathcal{A}_2$ is unforgeable, which means the proposed scheme is secure.

Table III presents a functional comparison between our scheme and the compared schemes, specifically in terms of identity authentication, conditional privacy protection, traceability, unlinkability, resistance to replay attacks, resistance to forgery, resistance to collusion attacks, batch authentication, avoidance of bilinear pairing operations, and context security. These functionalities are represented by S1–S10, respectively. Additionally, √ indicates that a certain function is achieved, while × indicates that it is either not considered or not implemented.

From Table III, we can see that schemes [18], [22], and [32] do not consider conditional privacy protection. Scheme [18] does not consider traceability, while scheme [37] lacks resistance to forgery. Additionally, schemes [18], [32], and [33] do not consider aggregate signatures. Although scheme [34] incorporates aggregate signatures, it does not consider resistance to collusion attacks. Furthermore, our scheme introduces context security, which is not present in other schemes. Therefore, our scheme provides more comprehensive functionality and higher security.

TABLE IV
PERFORMANCE SYMBOLS AND THEIR DESCRIPTIONS

Symbols	Description	Running time(ms)
T_M	Time to perform a scalar multiplication operation on $\mathbb{G}$	1.338
T_A	Time to perform a point addition operation on $\mathbb{G}$	0.01
T_P	Time to perform a bilinear pairing operation	9.88
T_H	Time to perform a map-to-point hash function operation	7.388
T_{MP}	Time to perform a scalar multiplication operation on $\mathbb{G}_1$	2.985
T_{MA}	Time to perform a point addition operation on $\mathbb{G}_1$	0.035

VI. PERFORMANCE ANALYSIS

This section provides the comparison with several relevant schemes [18], [22], [32], [33], [34], [37], in terms of computational cost, and communication overhead. For better experimentation and comparison, we take 1024-bit RSA, which corresponds to an 80-bit security level, as the benchmark. To achieve the security level of the 1024-bit RSA algorithm, for scheme [18], we employ a bilinear map $e : \mathbb{G}_1\times\mathbb{G}_1\rightarrow\mathbb{G}_2$ where $\mathbb{G}_1$ is a singular elliptic curve $E : y^2 = x^3\text{–}3x \bmod p$, with p being a secure large prime of 512 bits and the order q being a prime of 160 bits. For other schemes that do not utilize bilinear pairings [22], [32], [33], [34], [37], we select an elliptic curve $E : y^2 = x^3 + ax + b \bmod p$,where both p and q are large primes of 160 bits, and a, $b\in\mathbb{Z}_q^*$, to meet the security level of 1024-bit RSA, which is an 80-bit security level. The experiments were conducted by compiling the MIRACL cryptographic library using C++, with the testing environment being the Windows 10 operating system, an Intel Core i7-8750H CPU @ 2.20 GHz (2.21 GHz), and 16 GB of RAM. The specific computational cost and communication overhead of each scheme are shown in Tables V and VI.

A. Computational Cost

This section will conduct a performance comparison between the proposed scheme and other CLSs. For comparison purposes, the time to perform a scalar multiplication in $\mathbb{G}$ is denoted as T_M, and the time to perform a point addition operation in $\mathbb{G}$ is denoted as T_A. T_P represents the time required for a pairing operation. T_H represents the time required to execute a single map-to-point hash function.The time to perform a scalar multiplication operation on $\mathbb{G}_1$ is denoted as T_{MP}, and the time taken to perform a point addition operation on $\mathbb{G}_1$ is denoted as T_{MA}. Since the above operations

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 18,2026 at 09:20:19 UTC from IEEE Xplore. Restrictions apply.

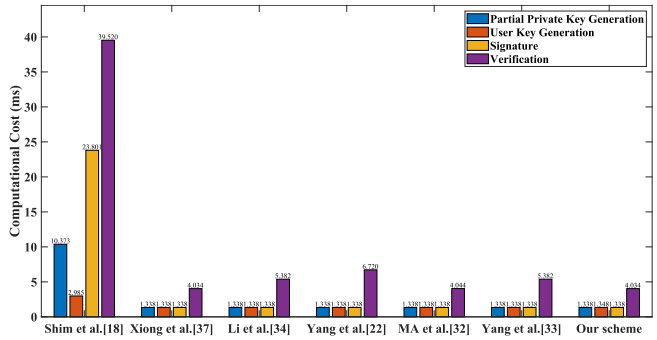

Fig. 3. Computational cost comparison of different schemes.

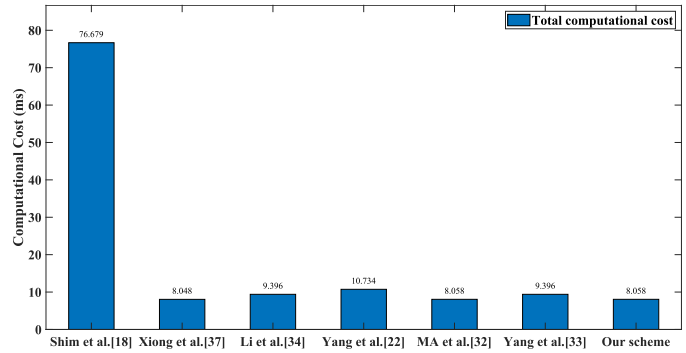

Fig. 4. Total computational cost comparison of different schemes.

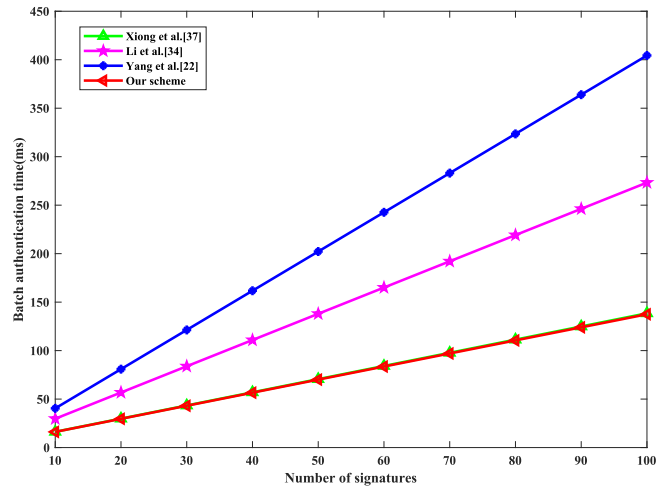

Fig. 5. Batch authentication computational cost comparison of different schemes.

have a significant impact on the verification efficiency of CLSs, this article only considers the time overhead of these operations. The specific time overhead of each operation is shown in Table IV.

As shown in Table V, Scheme [18] requires three scalar multiplication operations, two point addition operations, and two map-to-point hash operations during signing. During verification, it requires four pairing operations and three map-to-point hash operations. Therefore, the time overhead for signing and signature verification in Scheme [18] are $3T_{MP} + 2T_{MA} + 2T_H \approx 23.801$ ms and $4T_P \approx 39.52$ ms, respectively. Schemes [22], [32], [33], [34], [37], and our scheme all perform only one scalar multiplication operation during signing. Schemes [34] and [33] use three point addition operations and four scalar multiplication operations during verification, with a time overhead of $3T_A + 4T_M \approx 5.382$ ms. Scheme [22] uses three point addition operations and five scalar multiplication operations during signature verification, with a time overhead of $3T_A + 5T_M \approx 6.72$ ms. Scheme [32] performs three scalar multiplications and three point addition operations during signature verification. Therefore, the time overhead for signing and signature verification in Scheme [32] is $T_M \approx 1.338$ ms and $3T_A + 3T_M \approx 4.044$ ms, respectively. Scheme [37] and our scheme both perform two point addition operations and three scalar multiplications during signature verification. Therefore, the time overhead for signature verification in Scheme [37] and our scheme is $2T_A + 3T_M \approx 4.034$ ms. However, since Scheme [37] cannot resist attacks from arbitrary adversaries, it is not secure. Therefore, the scheme in this article ensures security while having a smaller overhead for signature generation at the user side and a lower computational cost for signature verification at the verifier side. Fig. 3 shows a comparison of the signature and verification time overheads for each scheme. Fig. 4 shows the total overhead of each scheme.

In batch verification of n signatures, Scheme [37] requires $2n$ point addition operations and $(n + 2)$ scalar multiplication operations, thus the time overhead for batch verification in Scheme [37] is $2nT_A + (n + 2)T_M \approx (1.358n+2.676)$ ms. Scheme [34] requires $3n$ point addition operations and $(2n + 2)$ scalar multiplication operations, so the time overhead for batch verification in Scheme [34] is $3nT_A + (2n + 2)T_M \approx (2.706n+2.676)$ ms. Scheme [22] requires $(3n-1)$ point addition operations and $3n$ scalar multiplication operations. Therefore, the time overhead for batch verification in Scheme [22] is $(3n - 1)T_A + 3nT_M \approx (4.044n-0.01)$ ms. In contrast, the scheme in this article requires only $(n + 1)$ point addition operations and $(n + 2)$ scalar multiplication operations for batch verification. Therefore, the time overhead for batch verification in the CLS in this article is $(n + 1)T_A + (n + 2)T_M \approx (1.348n+2.686)$ ms. As a result, the proposed scheme also exhibits high verification efficiency in batch verification. Fig. 5 shows a comparison of the time overhead for batch verification of each scheme with different numbers of signatures.

B. Communication Overhead

Since communication overhead is influenced by factors such as contextual fluctuations, user identity information, and message length, this article only considers the communication overhead of signatures and user public keys, the length of a single element in $\mathbb{G}_1$ is $64\times2 = 128$ bytes, that is, $|\mathbb{G}_1| = 128$ bytes. The length of $\mathbb{G}$ is $20\times2 = 40$ bytes, that is, $|\mathbb{G}| = 40$ bytes, and the single element in the finite field $|\mathbb{Z}_q^*|$ is 20 bytes. The comparison results of the communication overhead for each scheme are shown in Table VI, Figs. 6 and 7. As shown in Table VI, in Scheme [18], due to the use of a hash function mapping to a point during signature generation, the signature length is $2|\mathbb{G}_1| = 256$ bytes, the user public key length is $|\mathbb{G}_1| = 128$ bytes, and the total length is $3|\mathbb{G}_1| = 384$

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 18,2026 at 09:20:19 UTC from IEEE Xplore. Restrictions apply.

TABLE V
COMPUTATIONAL COST COMPARISON

Scheme	Partial Private Key Generation	User Key Generation	Signature	Verification(ms)	Batch Verification(ms)
Shim [18]	T_{MP}+T_H	T_{MP}	$3T_{MP}$+$2T_{MA}$+$2T_H$	$4T_P$≈39.52	-
Xiong [37]	T_M	T_M	T_M	$2T_A$+$3T_M$≈4.034	$2nT_A$+(n+2)T_M≈1.358n+2.676
Li [34]	T_M	T_M	T_M	$3T_A$+$4T_M$≈5.382	$3nT_A$+(2n+2)T_M≈2.706n+2.676
Yang [22]	T_M	T_M	T_M	$3T_A$+$5T_M$≈6.72	(3n-1)T_A+3nT_M≈4.044n-0.01
Ma [32]	T_M	T_M	T_M	$3T_A$+$3T_M$≈4.044	-
Yang [33]	T_M	T_M	T_M	$3T_A$+$4T_M$≈5.382	-
Our scheme	T_M	T_M+T_A	T_M	$2T_A$+$3T_M$≈4.034	(n+1)T_A+(n+2)T_M≈1.348n+2.686

TABLE VI
COMMUNICATION OVERHEAD COMPARISON

Scheme	Signature	User Public Key	Total Overhead(Byte)
Shim [18]	$2\|\mathbb{G}_1\|$	$\|\mathbb{G}_1\|$	$3\|\mathbb{G}_1\|$=384
Xiong [37]	$\|\mathbb{Z}_q^*\|$	$2\|\mathbb{G}\|$	$\|\mathbb{Z}_q^*\|$+$2\|\mathbb{G}\|$=100
Li [34]	$\|\mathbb{Z}_q^*\|$	$2\|\mathbb{G}\|$	$\|\mathbb{Z}_q^*\|$+$2\|\mathbb{G}\|$=100
Yang [22]	$\|\mathbb{Z}_q^*\|$+$\|\mathbb{G}\|$	$2\|\mathbb{G}\|$	$\|\mathbb{Z}_q^*\|$+$3\|\mathbb{G}\|$=140
Ma [32]	$\|\mathbb{Z}_q^*\|$+$\|\mathbb{G}\|$	$2\|\mathbb{G}\|$	$\|\mathbb{Z}_q^*\|$+$3\|\mathbb{G}\|$=140
Yang [33]	$\|\mathbb{Z}_q^*\|$+$\|\mathbb{G}\|$	$2\|\mathbb{G}\|$	$\|\mathbb{Z}_q^*\|$+$3\|\mathbb{G}\|$=140
Our scheme	$\|\mathbb{Z}_q^*\|$+$\|\mathbb{G}\|$	$\|\mathbb{G}\|$	$\|\mathbb{Z}_q^*\|$+$2\|\mathbb{G}\|$=100

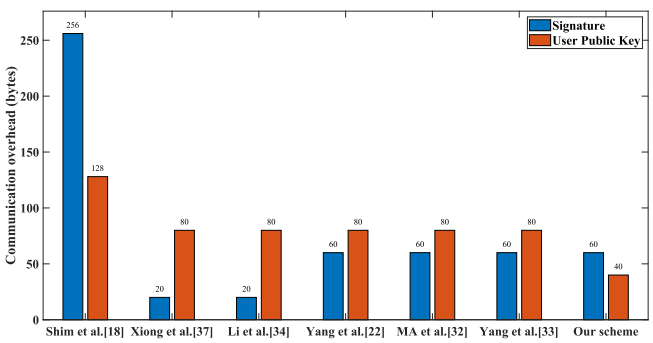

Fig. 6. Communication overhead comparison of different schemes.

bytes. Schemes [37] and [34] have signatures that consist of an element from the group $\mathbb{Z}_q^*$, so the signature length is $|\mathbb{Z}_q^*| =$ 20 bytes, and the public key similarly contains two elements from the elliptic curve. Thus, the total length is $|\mathbb{Z}_q^*|+2|\mathbb{G}| =$ 100 bytes. Similarly, in Schemes [22], [32], and [33], since the generated signature includes an element from the group $\mathbb{Z}_q^*$ and an element from the elliptic curve, the signature length is $|\mathbb{Z}_q^*|+|\mathbb{G}| = 60$ bytes, and the user public key contains two elements from the elliptic curve, so the public key length is $2|\mathbb{G}| = 80$ bytes. Therefore, the total length is $|\mathbb{Z}_q^*|+3|\mathbb{G}| =$ 140 bytes. In contrast, the signature in this article consists of an element from the group and an element from the elliptic curve, so the signature length is $|\mathbb{Z}_q^*|+|\mathbb{G}| = 60$ bytes, and the public key length is $|\mathbb{G}| = 40$ bytes. Therefore, the total length of the scheme in this article is $|\mathbb{Z}_q^*|+2|\mathbb{G}| = 100$ bytes. Thus, the CLS in this article has a lower communication overhead.

C. Energy Consumption

We analyze the energy consumption based on the evaluation approach proposed by Thumbur et al. [42]. The energy consumption can be calculated using the formula $E_c = t_c P_c$, where

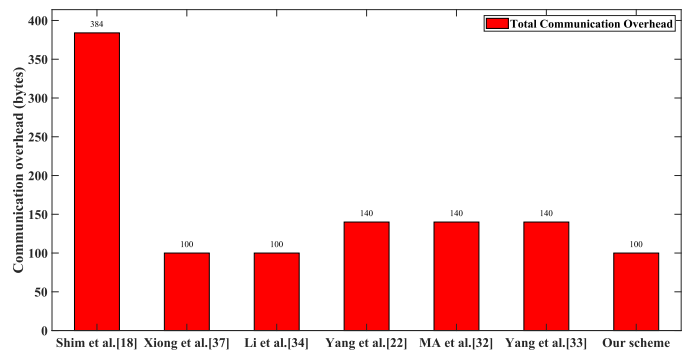

Fig. 7. Total communication overhead comparison of different schemes.

TABLE VII
ENERGY CONSUMPTION COMPARISON

Scheme	Signature Generation (mj)	Signature Verification (mj)	Batch Verification (mj)(n=100)
Shim [18]	251.14	429.02	42902.00
Xiong [37]	14.55	43.88	1507.98
Li [34]	14.55	58.54	2975.95
Yang [22]	14.55	73.02	4397.98
Ma [32]	14.55	43.98	4398.00
Yang [33]	14.55	58.54	5854.00
Our scheme	14.55	43.88	1495.25

E_c represents the energy consumption, t_c is the time required for signature generation or verification, and P_c denotes the maximum CPU power, which is 10.88 W.

Table VII presents the energy consumption of each scheme during signature generation, verification, and batch verification. As shown in the table, pairing-free schemes generally consume less energy across all three phases. Compared with other pairing-free schemes, our scheme requires the same amount of energy for signature generation. However, during signature verification, it consumes less energy than all other schemes except Scheme [37], which is known to be insecure. Notably, in the batch verification phase, our scheme demonstrates the lowest energy consumption. These results indicate that our scheme achieves a balanced tradeoff between security and energy efficiency.

VII. CONCLUSION

For the authentication problem in the healthcare scenario, this article proposes a multiparty authentication scheme

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 18,2026 at 09:20:19 UTC from IEEE Xplore. Restrictions apply.

for smart healthcare. The scheme combines blockchain and certificateless signature technology to build a smart healthcare system model. When a user needs to access the system, context-related devices obtain the user's contextual information, and the user must generate a user signature for identity authentication. After receiving the user's request, the system verifies whether the user can access the system by calling the relevant smart contracts. Finally, this article proves the security of the certificateless signature through security analysis and demonstrates its high efficiency in performance through performance analysis.

REFERENCES

[1] S. Tian, W. Yang, J. M. L. Grange, P. Wang, W. Huang, and Z. Ye, "Smart healthcare: Making medical care more intelligent," *Glob. Health J.*, vol. 3, no. 3, pp. 62–65, 2019.

[2] M. Poongodi, A. Sharma, M. Hamdi, M. Maode, and N. Chilamkurti, "Smart healthcare in smart cities: Wireless patient monitoring system using IoT," *J. Supercomput.*, vol. 77, pp. 12230–12255, Apr. 2021.

[3] R. F. Mansour, A. El Amraoui, I. Nouaouri, V. G. Díaz, D. Gupta, and S. Kumar, "Artificial intelligence and Internet of Things enabled disease diagnosis model for smart healthcare systems," *IEEE Access*, vol. 9, pp. 45137–45146, 2021.

[4] P. Kumar and H.-J. Lee, "Security issues in healthcare applications using wireless medical sensor networks: A survey," *Sensors*, vol. 12, no. 1, pp. 55–91, 2011.

[5] M. Rakhra, A. Singh, D. Singh, and Shruti, "Digital signature verification in cloud computing," in *Proc. 11th Int. Conf. Reliab., Infocom Technol. Optim. (Trends Future Directions) (ICRITO)*, 2024, pp. 1–6.

[6] A. J. Antony and K. Singh, "A blockchain-based public key infrastructure for IoT-based healthcare systems," *Comput. J.*, vol. 67, no. 4, pp. 1531–1537, 2024.

[7] A. Kumari, S. Bhaskar, S. Patil, and K. Parmar, "Decentralized and multi-authority based public key infrastructure for sharing electronic health records," *Procedia Comput. Sci.*, vol. 230, pp. 44–54, Jan. 2023.

[8] Y. Sun, R. Zhang, X. Wang, K. Gao, and L. Liu, "A decentralizing attribute-based signature for healthcare blockchain," in *Proc. 27th Int. Conf. Comput. Commun. Netw. (ICCCN)*, 2018, pp. 1–9.

[9] Q. Su, R. Zhang, R. Xue, and P. Li, "Revocable attribute-based signature for blockchain-based healthcare system," *IEEE Access*, vol. 8, pp. 127884–127896, 2020.

[10] J. Liu, H. Cao, Q. Li, F. Cai, X. Du, and M. Guizani, "A large-scale concurrent data anonymous batch verification scheme for mobile healthcare crowd sensing," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 1321–1330, Apr. 2019.

[11] Y. Xie, X. Li, S. Zhang, and Y. Li, "*iCLAS*: An improved certificateless aggregate signature scheme for healthcare wireless sensor networks," *IEEE Access*, vol. 7, pp. 15170–15182, 2019.

[12] A. A. Addobea, J. Hou, and Q. Li, "MHCOOS: An offline-online certificateless signature scheme for M-health devices," *Secur. Commun. Netw.*, vol. 2020, no. 1, 2020, Art. no. 7085623.

[13] N. B. Gayathri, G. Thumbur, P. R. Kumar, M. Z. U. Rahman, P. V. Reddy, and A. Lay-Ekuakille, "Efficient and secure pairing-free certificateless aggregate signature scheme for healthcare wireless medical sensor networks," *IEEE Internet Things J.*, vol. 6, no. 5, pp. 9064–9075, Oct. 2019.

[14] H. Du, Q. Wen, and S. Zhang, "An efficient certificateless aggregate signature scheme without pairings for healthcare wireless sensor network," *IEEE Access*, vol. 7, pp. 42683–42693, 2019.

[15] Y. Zhan, B. Wang, and R. Lu, "Cryptanalysis and improvement of a pairing-free certificateless aggregate signature in healthcare wireless medical sensor networks," *IEEE Internet Things J.*, vol. 8, no. 7, pp. 5973–5984, Apr. 2021.

[16] L. Deng, Y. Yang, and R. Gao, "Certificateless designated verifier anonymous aggregate signature scheme for healthcare wireless sensor networks," *IEEE Internet Things J.*, vol. 8, no. 11, pp. 8897–8909, Jun. 2021.

[17] L. Zhou and X. Yin, "An improved pairing-free certificateless aggregate signature scheme for healthcare wireless medical sensor networks," *PLoS One*, vol. 17, no. 7, 2022, Art. no. e0268484.

[18] K.-A. Shim, "A secure certificateless signature scheme for cloud-assisted industrial IoT," *IEEE Trans. Ind. Informat.*, vol. 20, no. 4, pp. 6834–6843, Apr. 2024.

[19] W. Wang, H. Xu, M. Alazab, T. R. Gadekallu, Z. Han, and C. Su, "Blockchain-based reliable and efficient certificateless signature for IIoT devices," *IEEE Trans. Ind. Informat.*, vol. 18, no. 10, pp. 7059–7067, Oct. 2022.

[20] C. He, B. Zhang, L. Zhang, Z. Xi, Y. Fang, and Y. Wang, "Improving throughput of mobile sensors via certificateless signature supporting batch verification," *Electronics*, vol. 12, no. 22, p. 4700, 2023.

[21] Q. Wu, L. Zhang, Y. Yang, and K.-K. R. Choo, "Certificateless signature scheme with batch verification for secure and privacy-preserving V2V communications in VANETs," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 2, pp. 1448–1459, Mar./Apr. 2025.

[22] X. Yang, H. Wen, R. Diao, X. Du, and C. Wang, "Improved security of a pairing-free certificateless aggregate signature in healthcare wireless medical sensor networks," *IEEE Internet Things J.*, vol. 10, no. 12, pp. 10881–10892, Jun. 2023.

[23] S. S. Al-Riyami and K. G. Paterson, "Certificateless public key cryptography," in *Proc. 9th Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, 2003, pp. 452–473.

[24] K.-A. Shim, "A new certificateless signature scheme provably secure in the standard model," *IEEE Syst. J.*, vol. 13, no. 2, pp. 1421–1430, Jun. 2019.

[25] X. Yang, H. Wen, L. Liu, N. Ren, and C. Wang, "Blockchain-enhanced certificateless signature scheme in the standard model," *Math. Biosci. Eng.*, vol. 20, no. 7, pp. 12718–12730, 2023.

[26] H. Wang, L. Wang, K. Zhang, J. Li, and Y. Luo, "A conditional privacy-preserving certificateless aggregate signature scheme in the standard model for VANETs," *IEEE Access*, vol. 10, pp. 15605–15618, 2022.

[27] X. Yang, R. Diao, T. Liu, H. Wen, and C. Wang, "Electricity carbon quota trading scheme based on certificateless signature and blockchain," *CMES-Comput. Model. Eng. Sci.*, vol. 138, no. 2, pp. 1695–1712, 2024.

[28] H.-Y. Lin, T.-T. Tsai, and H.-L. Wu, "A revocable certificateless sanitizable signature scheme with batch verification," *IEEE Access*, vol. 12, pp. 143392–143400, 2024.

[29] X. Jia, D. He, Q. Liu, and K.-K. R. Choo, "An efficient provably-secure certificateless signature scheme for Internet-of-Things deployment," *Ad Hoc Netw.*, vol. 71, pp. 78–87, Mar. 2018.

[30] H. Du, Q. Wen, S. Zhang, and M. Gao, "A new provably secure certificateless signature scheme for Internet of Things," *Ad Hoc Netw.*, vol. 100, Apr. 2020, Art. no. 102074.

[31] D. Xiang, X. Li, J. Gao, and X. Zhang, "A secure and efficient certificateless signature scheme for Internet of Things," *Ad Hoc Netw.*, vol. 124, Jan. 2022, Art. no. 102702.

[32] K. Ma et al., "An efficient certificateless signature scheme with provably security and its applications," *IEEE Syst. J.*, vol. 17, no. 4, pp. 5636–5647, Dec. 2023.

[33] X. Yang, W. Wang, T. Tian, and C. Wang, "Cryptanalysis and improvement of a blockchain-based certificateless signature for IIoT devices," *IEEE Trans. Ind. Informat.*, vol. 20, no. 2, pp. 1884–1894, Feb. 2024.

[34] X. Li, X. Yin, and J. Ning, "RelCLAS: A reliable malicious KGC-resistant certificateless aggregate signature protocol for vehicular ad hoc networks," *IEEE Internet Things J.*, vol. 10, no. 23, pp. 21100–21114, Dec. 2023.

[35] S. Liu, Z. Wan, Y. Yuan, Q. Dong, B. Yang, and F. Hao, "Efficient certificateless blind signature scheme with conditional revocation for mobile crowdsensing within smart city," *IEEE Internet Things J.*, vol. 11, no. 9, pp. 15985–15997, May 2024.

[36] B. Chen, T. Xiang, X. Li, M. Zhang, and D. He, "Efficient attribute-based signature with collusion resistance for Internet of Vehicles," *IEEE Trans. Veh. Technol.*, vol. 72, no. 6, pp. 7844–7856, Jun. 2023.

[37] W. J. Xiong, R. M. Wang, Y. J. Wang, F. Zhou, and X. N. Luo, "A conditional privacy-preservingbatch authentication scheme based on certificateless aggregate signature for VANETs," *J. Cryptologic Res.*, vol. 10, no. 3, p. 462, 2023.

[38] V. I. Nechaev, "Complexity of a determinate algorithm for the discrete logarithm," *Math. Notes*, vol. 55, no. 1, pp. 165–172, 1994.

[39] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," in *Proc. Decentralized Bus. Rev.*, 2008, Art. no. 21260.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 18,2026 at 09:20:19 UTC from IEEE Xplore. Restrictions apply.

[40] N. Szabo, "Smart contracts: Building blocks for digital markets," *EXTROPY, J. Transhumanist Thought*, vol. 18, no. 2, p. 28, 1996.
[41] M. A. Saleem, X. Li, M. F. Ayub, S. Shamshad, F. Wu, and H. Abbas, "An efficient and physically secure privacy-preserving key-agreement protocol for vehicular ad-hoc network," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 9, pp. 9940–9951, Sep. 2023.
[42] G. Thumbur, G. S. Rao, P. V. Reddy, N. Gayathri, D. K. R. K. Reddy, and M. Padmavathamma, "Efficient and secure certificateless aggregate signature-based authentication scheme for vehicular ad hoc networks," *IEEE Internet Things J.*, vol. 8, no. 3, pp. 1908–1920, Feb. 2021.

Yousheng Zhou received the Ph.D. degree in cryptography from the School of Computer Science, Beijing University of Posts and Telecommunications, Beijing, China, in 2011.

His main research interests include vehicular network security, privacy computing, artificial intelligence security, and network attack and defense.

Longjie Li is currently pursuing the M.S. degree with Chongqing University of Posts and Telecommunications, Chongqing, China.

His research interests include computer networks and system security.

Xiangjian Zuo received the Ph.D. degree in information security from the School of Cyberspace Security, Beijing University of Posts and Telecommunications, Beijing, China, in 2021.

His main research interests include secure multiparty computation and data security and privacy protection.

Yuanni Liu received the Ph.D. degree in computer science and technology from the Institute of Network Technology, Beijing University of Posts and Telecommunications, Beijing, China, in 2011.

Her main research interests include mobile crowd sensing, IoT security, and IP routing technology.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 18,2026 at 09:20:19 UTC from IEEE Xplore. Restrictions apply.